

Fase 2: Consumer

Objetivo

Validar de forma exhaustiva que, vía API, un consumidor puede:

- autenticarse;
- localizar participantes o conectores relevantes;
- consultar catálogo remoto y/o federado;
- recuperar offers útiles asociadas a assets;
- interpretar las condiciones del contrato;
- y dejar identificado el contrato o acuerdo necesario para pasar a transfer.

Alcance

Esta fase cubre **solo consumer**.

No entra todavía en la transferencia como tal, salvo la comprobación mínima de que la salida de esta fase deja preparado el `contractId` o el acuerdo necesario para la fase 3. La documentación sí deja claro que después vendrá `transferprocesses`, usando `contractId`, `assetId`, `counterPartyAddress` y `protocol`.

Aclaración importante

Aquí conviene dejar una cuestión clara: en la documentación disponible **sí aparece con claridad el flujo funcional de negociación**, y en interfaz se observa un botón de negociar contrato, pero **todavía no se ha localizado en estos PDFs un endpoint explícito y documentado de contract negotiation / contract agreement** como sí aparece para catálogo o transferencia.

Lo que sí está documentado es que existen **contract definitions**, que el catálogo expone **ofertas**, y que el sistema cuenta entidades de tipo **contractAgreement** en el Count API. Así que esta parte es necesario plantearla como **validación con un hueco documental pendiente de confirmar en la colección real o en el OpenAPI completo**.

Dicho de otra forma: la fase se puede planificar perfectamente, pero el paso exacto de "cerrar acuerdo" por API necesita confirmar el endpoint concreto en la colección o en el conector.

Propuesta cerrada: Fase 2 Consumer por API

Bloque 0. Prechecks

0.1. Autenticación

Igual que en provider, el punto de entrada es Keycloak, realm `dataspace`, client `dataspace-users`, y después todas las llamadas relevantes van con `Authorization: Bearer <JWT>`. Esto aplica tanto a catálogo como a management y otros endpoints protegidos.

0.2. Descubrimiento de participantes

Desde el rol de consumer, antes de consultar catálogo conviene identificar a qué conector consultar. El Registration Service mantiene el registro de participantes y expone `GET /participants` y `GET /public/participants`, con `participantId` y `url` del conector. Eso sirve para construir o validar el `counterPartyAddress` del provider.

0.3. Conector operativo

El documento también deja claro que, aunque falte UI, el conector sigue siendo funcional por API. En cambio, si el conector falla, no hay consumo real posible. Por tanto, este precheck no es cosmético: es una condición de viabilidad de toda la fase.

Bloque 1. Descubrimiento del catálogo

Este es el primer bloque fuerte de consumer. El conector expone dos formas de descubrir recursos:

- **catálogo remoto** de otro conector;
- **catálogo federado** cacheado en el Federated Catalog.

Endpoints base

Catálogo remoto

- `POST /management/v3/catalog/request`

Catálogo federado

- `POST /federatedcatalog/v1alpha/catalog/query`

Catálogo federado vía management

- `POST /management/federatedcatalog/request`

Qué validar

Consulta remota directa

- usar `counterPartyAddress` del provider y `protocol = dataspace-protocol-http`;
- comprobar que responde;
- verificar si devuelve assets y offers del provider esperado.

Consulta federada

- lanzar una query global;
- comprobar si el asset del provider aparece en el catálogo federado;
- comparar si lo observado coincide con el catálogo remoto o si hay desfase.

Paginación / filtros

- probar `QuerySpec` en `management/federatedcatalog/request`;
- validar que se pueden aislar ofertas por provider, tipo de asset o cualquier criterio disponible.

Qué se busca confirmar aquí

No solo que "hay datos", sino estas tres cosas:

- que el recurso es **descubrible**;
- que el catálogo devuelve **ofertas** y no solo assets desnudos;
- y que el consumidor puede localizar el recurso correcto sin depender de la UI.

Bloque 2. Inspección de assets y offers

En interfaz se observa que el Catalog Browser combina asset + ofertas + política. Por API, aquí es necesario reproducir exactamente eso, pero leyendo las respuestas del catálogo. Funcionalmente, el catálogo es la puerta de entrada al consumo operativo.

Qué validar

Asset

- identificar `assetId` ;
- comprobar si el metadata útil está presente;
- verificar si hay suficiente contexto para saber qué se está consumiendo.

Offers

- comprobar cuántas ofertas devuelve cada asset;
- identificar la offer adecuada para el caso de prueba;
- ver si las condiciones están suficientemente representadas en la respuesta.

Política

- si el catálogo devuelve la policy o una referencia suficiente, comprobar que se puede interpretar qué permite y qué obliga. En la documentación funcional, el contrato se apoya en una Access Policy y una Contract Policy reutilizables, y las demos usan ejemplos como `PURPOSE = research` , permiso `USE` y obligación `NOTIFY` . Eso sirve como oráculo conceptual para interpretar la oferta recibida.

Casos a probar

- un asset con una sola oferta;
- un asset con varias ofertas;
- un asset de tipo dataset;
- un asset de tipo service;
- un asset de tipo machine learning, para ver si el metadata o la semántica asociada cambia. El sistema contempla esos tres tipos como tipologías principales.

Bloque 3. Negociación / acuerdo

Aquí está la parte delicada. Funcionalmente, el flujo existe: el consumidor consulta catálogo, selecciona asset u oferta, inicia la negociación y después se establece el acuerdo. Eso está documentado en las notas funcionales y es coherente con el comportamiento observado en UI.

Pero, a nivel de API, en la documentación disponible **todavía no aparece el endpoint explícito** de esa negociación. Así que este bloque se divide en dos subniveles:

3.1. Validación funcional

Objetivo: confirmar qué oferta se va a intentar contratar y con qué identificadores.

Deben quedar registrados:

- `assetId`
- identificador de la offer o dato equivalente
- `counterPartyAddress`
- `protocol`
- provider seleccionado

3.2. Validación técnica pendiente

Objetivo: localizar el endpoint real de negociación/acuerdo en la colección o en el OpenAPI completo.

Hasta que eso aparezca, no conviene afirmar un path concreto. Sí puede prepararse la batería de pruebas y usar como comprobación indirecta que el sistema expone conteo para `contractAgreement`, lo que indica que esa entidad existe y es observable desde management.

Pruebas que deben planificarse

Happy path

- iniciar negociación sobre una oferta válida;
- verificar si se genera agreement o identificador equivalente.

Negativa

- intentar negociar una offer inválida o incompleta;
- repetir la negociación de la misma offer;
- usar un `counterPartyAddress` erróneo;
- usar un `assetId` correcto con una offer que no le corresponde.

Verificación

- contar `contractAgreement` antes y después;
 - recuperar el agreement o evidencia equivalente si existe endpoint;
 - comprobar si ya es posible pasar a transfer con un `contractId` usable.
-

Bloque 4. Verificación post-negociación

Aunque la transferencia venga en la fase 3, aquí debe quedar confirmada la salida de consumer.

Qué debe salir de esta fase

Como mínimo, la fase debería terminar con esto:

- el recurso correcto identificado;
- la offer correcta identificada;
- el provider y `counterPartyAddress` confirmados;
- el `assetId` que se va a transferir;
- y, si el endpoint de negociación está localizado y funciona, el `contractId` o acuerdo resultante.

Oráculos útiles

Count API

- `POST /pagination/count?type=contractAgreement`
- `POST /pagination/count?type=federatedCatalog`

Sirve para medir si hay cambios reales en estado, aunque el catálogo o la negociación no den una respuesta muy expresiva.

Catálogo

- reconsultar el catálogo y comprobar si la offer o el recurso han cambiado de estado o siguen visibles igual.
-

Bloque 5. Preparación explícita para Transfer

Esta fase consumer por API debe terminar dejando todo listo para fase 3. La documentación de transfer deja muy claro qué necesita el request:

- `connectorId`

- `counterPartyAddress`
- `contractId`
- `assetId`
- `protocol`
- `transferType`
- opcionalmente `dataDestination` y `managedResources`

Por tanto, el cierre fuerte de esta fase no es “el catálogo se ha consultado”, sino:

ya quedan disponibles todos los identificadores y referencias necesarias para lanzar `transferprocesses` en la siguiente fase.

Orden recomendado de ejecución

Se recomienda este orden:

Ronda 1. Smoke test

- login como consumer;
- `GET /public/participants` ;
- una query al catálogo remoto;
- una query al catálogo federado;
- localizar un asset y su offer.

Ronda 2. Descubrimiento exhaustivo

- varios providers;
- varios assets;
- remoto vs federado;
- comprobación de consistencia.

Ronda 3. Inspección profunda

- metadata;

- offers;
- interpretación de condiciones;
- clasificación de qué offer vale para qué caso.

Ronda 4. Negociación

- localizar el endpoint real en la colección;
- ejecutar happy path;
- ejecutar negativas;
- verificar aparición de agreement o cambio de estado.

Ronda 5. Preparación de Transfer

- fijar `contractId`, `assetId`, `counterPartyAddress` y `protocol`;
 - dejar un caso listo para fase 3.
-

Qué sería “probar absolutamente todo” en Consumer por API

Aquí se pueden usar siete preguntas:

Descubrir

¿el recurso aparece?

Comparar

¿sale igual en catálogo remoto y federado?

Inspeccionar

¿se puede leer el asset y sus offers con suficiente contexto?

Interpretar

¿las condiciones de uso quedan claras?

Negociar

¿se puede iniciar el acuerdo desde API?

Verificar

¿el acuerdo queda reflejado en el sistema?

Preparar

¿la fase termina con todo lo necesario para transfer?

Registro de incidencias recomendado

Como en provider, se recomienda una ficha muy ligera:

Módulo

Auth / Participants / Remote Catalog / Federated Catalog / Offers / Negotiation

Caso de prueba

nombre corto

Precondición

qué asset u offer debía existir

Endpoint

ruta exacta

Payload

resumen

Esperado

qué debía pasar

Observado

qué pasó

Clasificación

bug / limitación / integración / hueco documental

Severidad

bloqueante / alta / media / baja

IDs clave

provider / asset / offer / contractId si existe

Evidencia

respuesta HTTP o log

Recomendación práctica

Se recomienda empezar con **un único provider conocido** y **un único asset bien controlado** creado en Fase 1. Camino mínimo:

1. listar participantes;
2. tomar `url` del provider;
3. `catalog/request` contra ese provider;
4. localizar el asset;
5. identificar su offer;
6. localizar el endpoint real de negociación en la colección;
7. ejecutar negociación;
8. dejar listo el `contractId` para transfer.

Lo más importante aquí no es solo “consultar catálogo”, sino **cerrar el puente entre catálogo y transferencia**. Esa es la salida funcional de Consumer.

Comandos del taller

Esta sección cierra el “hueco documental” mencionado más arriba: contiene los `curl` exactos para descubrir catálogo, capturar la oferta, lanzar la negociación y obtener el `contractAgreementId`. Se asume que la Fase 1: Provider ha terminado y que las variables de la Taller APIs - Plan de ejecución están cargadas en la terminal.

- `dataspace`
- `suffix`
- `provider`
- `consumer`
- `data env`
- `phase1`

2.1 Login del consumer

Mismo flujo de Keycloak que en la Fase 1, pero con las credenciales del conector consumer (`conn-company-demo`).

```
export CONSUMER_JWT=$(curl -s -X POST \
  "${KEYCLOAK_URL}/realms/${DS_NAME}/protocol/openid-connect/token" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "grant_type=password" \
  -d "client_id=${KC_CLIENT}" \
  -d "username=${CONSUMER_USER}" \
  -d "password=${CONSUMER_PASSWORD}" \
  -d "scope=openid profile email" | jq -r .access_token)

echo "CONSUMER_JWT: ${CONSUMER_JWT:0:40}..."
```

2.2 Catálogo remoto: el consumer descubre la oferta del provider

El consumer hace una `CatalogRequest` por DSP contra el endpoint **interno** del provider. La respuesta se guarda en una variable para poder extraer después `participantId`, `offer policyId` y `assetId`.

```
CATALOG_RESPONSE=$(curl -s -X POST "${CONSUMER_BASE}/management/v3/catalog/request" \
  -H "Authorization: Bearer ${CONSUMER_JWT}" \
  -H "Content-Type: application/json" \
  -d "$(cat <<EOF
{
  "@context": { "@vocab": "https://w3id.org/edc/v0.0.1/ns/"
},
  "@type": "CatalogRequest",
  "counterPartyAddress": "${PROVIDER_PROTOCOL}",
  "counterPartyId": "${PROVIDER}",
  "protocol": "dataspace-protocol-http",
  "querySpec": { "offset": 0, "limit": 100, "filterExpression": [] }
}
EOF
)")
```

```
echo "${CATALOG_RESPONSE}" | jq '. "dcat:dataset"'
```

Como resultado debe verse lo mismo que antes (Paso 1.8).

Captura de los identificadores clave para la negociación. Este `jq` filtra ofertas **sin** `odrl:obligation` y maneja `dcat:dataset` tanto si es objeto como si es array, así que es robusto incluso cuando hay ofertas residuales con `NOTIFY` en el catálogo:

```
export PROVIDER_PARTICIPANT_ID=$(echo "${CATALOG_RESPONSE}"  
| jq -r '. "dspace:participantId" // .participantId // "'${P  
ROVIDER}'"')
```

```
export OFFER_POLICY_ID=$(echo "${CATALOG_RESPONSE}" | jq -r  
,  
  ". "dcat:dataset"  
| (if type=="array" then . else [.] end)  
| map(select(  
  ". "odrl:hasPolicy"  
  | (if type=="array" then .[0] else . end)  
  | (. "odrl:obligation" // [])  
  | (if type=="array" then length else 1 end) == 0  
))  
| .[0]  
| ". "odrl:hasPolicy"  
| (if type=="array" then .[0] else . end)  
| ". @id"  
' )
```

```
export CATALOG_ASSET_ID=$(echo "${CATALOG_RESPONSE}" | jq -  
r '  
  ". "dcat:dataset"  
| (if type=="array" then . else [.] end)  
| map(select(  
  ". "odrl:hasPolicy"  
  | (if type=="array" then .[0] else . end)  
  | (. "odrl:obligation" // [])  
  | (if type=="array" then length else 1 end) == 0  
  )
```

```

    ))
    | .[0]
    | ."@id"
  ' )

echo "PROVIDER_PARTICIPANT_ID: ${PROVIDER_PARTICIPANT_ID}"
echo "OFFER_POLICY_ID:          ${OFFER_POLICY_ID}"
echo "CATALOG_ASSET_ID:         ${CATALOG_ASSET_ID}"

```



AVISO: Si `OFFER_POLICY_ID` o `CATALOG_ASSET_ID` salen `null`, normalmente significa que **todas** las ofertas del catálogo tienen `obligation` (sesión antigua). En ese caso, volver a la Fase 1: Provider y revisar el paso 1.5: la Contract Policy debe llevar `obligation: []`.

2.3 Catálogo federado (alternativa, mismo recurso)

Vista agregada del catálogo. El asset que publicó el provider en la Fase 1 debería aparecer aquí con el mismo `@id`.

```

curl -s -X POST "${CONSUMER_BASE}/management/federatedcatalog/request" \
  -H "Authorization: Bearer ${CONSUMER_JWT}" \
  -H "Content-Type: application/json" \
  -d '{"@context":{"@vocab":"https://w3id.org/edc/v0.0.1/ns/"},"offset":0,"limit":50,"filterExpression":[]}' \
  | jq '.[]
    | (. "dcat:dataset" // . "http://www.w3.org/ns/dcat#dataset" // empty)
    | (if type=="array" then . else [.] end)
    | .[]?
    | {id: ."@id", type: ."@type"}'

```

Este comando contempla ambos formatos de clave (`dcat:dataset` y URI completa).

2.4 Iniciar la negociación contractual

Con `OFFER_POLICY_ID`, `CATALOG_ASSET_ID` y `PROVIDER_PARTICIPANT_ID` ya capturados, se lanza la `ContractRequest` por DSP. La policy enviada en el body **debe replicar la oferta del catálogo** (en este taller, `permission: USE`); en cuanto el provider firme el agreement, el consumer compara su offer con el agreement firmado, así que cualquier diferencia provoca `state=TERMINATED`.

```
NEG_RESPONSE=$(curl -s -X POST "${CONSUMER_BASE}/management/v3/contractnegotiations" \
  -H "Authorization: Bearer ${CONSUMER_JWT}" \
  -H "Content-Type: application/json" \
  -d "$(cat <<EOF
{
  "@context": { "@vocab": "https://w3id.org/edc/v0.0.1/ns/"
},
  "@type": "ContractRequest",
  "counterPartyAddress": "${PROVIDER_PROTOCOL}",
  "counterPartyId": "${PROVIDER_PARTICIPANT_ID}",
  "protocol": "dataspace-protocol-http",
  "policy": {
    "@context": "http://www.w3.org/ns/odrl.jsonld",
    "@type": "Offer",
    "@id": "${OFFER_POLICY_ID}",
    "assigner": "${PROVIDER_PARTICIPANT_ID}",
    "target": "${CATALOG_ASSET_ID}",
    "permission": [ { "action": "USE" } ],
    "prohibition": [],
    "obligation": []
  }
}
EOF
)")

echo "${NEG_RESPONSE}" | jq .
export NEG_ID=$(echo "${NEG_RESPONSE}" | jq -r '["@id"]')
echo "NEG_ID: ${NEG_ID}"
```

Dentro de `policy` se replica la oferta del catálogo: `permission` con `action: USE`, y `prohibition` / `obligation` vacíos. Así la policy enviada coincide exactamente con la

firmada por el provider y la negociación llega a `FINALIZED`.



ERROR: `state=TERMINATED` sin `AGREEMENT_ID`

Causas habituales y cómo resolver:

1. **Mismatch de policy.** El provider firma el agreement con la policy de su Contract Definition (`permission: USE`), pero el consumer envió `permission: []`. En logs del consumer aparece: `Policy in the contract agreement is not equal to the one in the contract offer`. Solución: enviar `permission: [ { "action": "USE" } ]` como en el bloque de arriba, no array vacío.
2. **Oferta residual con `NOTIFY` en el catálogo.** Si la Fase 1 se ejecutó con `obligation: NOTIFY` en algún momento, esa oferta sigue colgada en el catálogo y bloquea la negociación.
3. **JWT caducado.** Los tokens de Keycloak expiran en pocos minutos. Si la respuesta es HTML 401/502 en lugar de JSON, renovar `CONSUMER_JWT` (paso 2.1) y volver a lanzar `ContractRequest`. Es importante exportar de nuevo `NEG_ID` con la respuesta nueva, no reutilizar el de la negociación terminada.

SOLUCIÓN: borrar la contract definition vieja y refrescar el catálogo:

```
curl -s -X POST "${PROVIDER_BASE}/management/v3/contractdefinitions/request" \
  -H "Authorization: Bearer ${PROVIDER_JWT}" \
  -H "Content-Type: application/json" \
  -d '{"@context":{"@vocab":"https://w3id.org/edc/v0.0.1/ns/"},"offset":0,"limit":50,"filterExpression":[]}' \
  | jq '.[] | {id: "@id", access: .accessPolicyId, contract: .contractPolicyId}'
```

```
curl -s -w "\nHTTP %{http_code}\n" -X DELETE "${PROVIDER_BASE}/management/v3/contractdefinitions/<CD_ID_VIEJA>" \
  -H "Authorization: Bearer ${PROVIDER_JWT}"
```

2.5 Polling del estado de la negociación hasta obtener `contractAgreementId`

La negociación es asíncrona: pasa por estados intermedios (`REQUESTED` , `AGREED` , `FINALIZED`) hasta que el provider responde con un agreement firme. El bucle ejecuta `GET` cada 3 segundos hasta encontrar el `contractAgreementId` o detectar `TERMINATED` .

```
export AGREEMENT_ID=""

for i in $(seq 1 20); do
    STATE_RESPONSE=$(curl -s "${CONSUMER_BASE}/management/v3/contractnegotiations/${NEG_ID}" \
        -H "Authorization: Bearer ${CONSUMER_JWT}")
    STATE=$(echo "${STATE_RESPONSE}" | jq -r '.state // ."edc:state"')
    AGREEMENT_ID=$(echo "${STATE_RESPONSE}" | jq -r '.contractAgreementId // ."edc:contractAgreementId" // empty')
    echo "[$i] state=${STATE} agreement=${AGREEMENT_ID:-<none>}"
    if [ -n "${AGREEMENT_ID}" ] && [ "${AGREEMENT_ID}" != "null" ]; then break; fi
    if [ "${STATE}" = "TERMINATED" ]; then echo "Negociación TERMINATED"; break; fi
    sleep 3
done

echo "AGREEMENT_ID final: ${AGREEMENT_ID}"
```

2.6 Verificar el agreement creado

Lectura directa del agreement por su id, para inspeccionar las fechas y la política firmada antes de pasar a Transfer.

```
curl -s "${CONSUMER_BASE}/management/v3/contractagreements/${AGREEMENT_ID}" \
    -H "Authorization: Bearer ${CONSUMER_JWT}" | jq .
```

En la respuesta, `assetId` es el nombre del asset relacionado con lo mostrado anteriormente, donde este valor es el que aparece en el catalog browser. Finalmente, abajo aparecen la fecha de firma y los participantes del acuerdo.

2.7 Resumen de IDs generados en la Fase 2

Cómoda referencia para arrancar la Fase 3 con todo lo necesario para `transferprocesses`.

```
cat <<EOF
```

```
===== FASE 2 - CONSUMER OK =====
PROVIDER_PARTICIPANT_ID: ${PROVIDER_PARTICIPANT_ID}
OFFER_POLICY_ID:         ${OFFER_POLICY_ID}
CATALOG_ASSET_ID:        ${CATALOG_ASSET_ID}
NEG_ID:                  ${NEG_ID}
AGREEMENT_ID:            ${AGREEMENT_ID}
=====

EOF
```

```
{
  printf 'export PROVIDER_PARTICIPANT_ID=%q\n' "$PROVIDER_P
ARTICIPANT_ID"
  printf 'export OFFER_POLICY_ID=%q\n' "$OFFER_POLICY_ID"
  printf 'export CATALOG_ASSET_ID=%q\n' "$CATALOG_ASSET_ID"
  printf 'export NEG_ID=%q\n' "$NEG_ID"
  printf 'export AGREEMENT_ID=%q\n' "$AGREEMENT_ID"
} > "/Users/jpardo/Desktop/Proyectos/INESdata/data_spaces/d
emos/API/phase2_env.sh"
```